Capítulo 1

Desarrollo de plugins

1.1. Introducción

SamaruC Editor incluye un sistema de plugins basado en **Java ServiceLoader** que permite extender la aplicación sin tocar su código fuente. Este documento explica paso a paso cómo crear, empaquetar e instalar un plugin.

1.2. Arquitectura general

La arquitectura de plugins se basa en una interfaz común `Plugin` que los desarrolladores implementan. SamaruC descubre e instancia estas clases automáticamente al arrancar, llamando a sus métodos de ciclo de vida para integrarlas en la aplicación, en la siguiente figura podemos ver un diagrama de la arquitectura del sistema de plugins.

```
+---------------------------------------------+
|            SamaruC Editor                   |
|                                             |
|  PluginManager                              |
|   +-- escanea carpeta plugins/              |
|   +-- carga cada JAR con URLClassLoader     |
|   +-- descubre implementaciones via         |
|   |   ServiceLoader<Plugin>                 |
|   +-- llama a onLoad -> onEnable            |
|                                             |
|  PluginContext  (pasado a cada plugin)      |
|   +-- getPrimaryStage()                     |
|   +-- getScene()                            |
|   +-- getMenuBar()                          |
|   +-- log(message)                          |
|   +-- registerMenu(menu)                    |
|   +-- unregisterMenu(menu)                  |
+---------------------------------------------+
         ^
         |  implements
+-----------------+
|   Tu plugin     |
|  (Plugin.java)  |
+-----------------+
```

Figura 1.1: Diagrama de arquitectura del sistema de plugins.

El ciclo de vida de un plugin se describe en la siguiente tabla:

Método	**Cuándo se llama**
`onLoad(ctx)`	Justo después de cargar el JAR. Guarda el contexto aquí.
`onEnable()`	Cuando la UI está lista. Agrega tus menús/componentes aquí.
`onDisable()`	Cuando el plugin se desactiva o la app cierra. Limpia recursos aquí.
`getName()`	En cualquier momento. Devuelve el nombre visible del plugin.

Tabla 1.1: Ciclo de vida de un plugin

1.3. Requisitos previos

- **JDK 17** o superior
- **Maven 3.8+** (o Gradle si lo prefieres)
- El JAR principal de SamaruC (`retro-editor-X.Y-SNAPSHOT.jar`) para compilar contra sus interfaces

Consejo: Añade el JAR de SamaruC a tu repositorio Maven local con:

```
mvn install:install-file -Dfile=retro-editor-1.0-SNAPSHOT.jar \
  -DgroupId=com.retroeditor -DartifactId=retro-editor \
  -Dversion=1.0-SNAPSHOT -Dpackaging=jar
```

1.4. Crear el proyecto Maven del plugin

Crea la siguiente estructura de directorios:

```
mi-plugin/
+-- pom.xml
+-- src/
    +-- main/
        +-- java/
        |   +-- com/
        |       +-- miplugin/
        |           +-- MiPlugin.java
        +-- resources/
            +-- META-INF/
                +-- services/
                    +-- com.retroeditor.plugin.Plugin
```

Figura 1.2: Estructura de directorios del proyecto Maven del plugin.

`pom.xml` mínimo:

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
             http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>com.miplugin</groupId>
    <artifactId>mi-plugin</artifactId>
    <version>1.0.0</version>
    <packaging>jar</packaging>

    <dependencies>
        <!-- Interfaces de SamaruC (solo compile-time) -->
        <dependency>
            <groupId>com.retroeditor</groupId>
            <artifactId>retro-editor</artifactId>
            <version>1.0-SNAPSHOT</version>
            <scope>provided</scope>
        </dependency>

        <!-- JavaFX (solo compile-time, lo provee el host) -->
        <dependency>
            <groupId>org.openjfx</groupId>
            <artifactId>javafx-controls</artifactId>
            <version>17.0.10</version>
            <scope>provided</scope>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-compiler-plugin</artifactId>
                <version>3.8.1</version>
```

```
                <configuration>
                    <source>17</source>
                    <target>17</target>
                </configuration>
            </plugin>
        </plugins>
    </build>
</project>
```

Listing 1.1: Configuración Maven básica

Importante: Las dependencias de SamaruC y JavaFX deben declararse con `<scope>provided</scope>` porque ya están disponibles en el classpath del host en tiempo de ejecución.

1.5. Implementar la interfaz `Plugin`

Crea `MiPlugin.java` implementando `com.retroeditor.plugin.Plugin`:

```java
package com.miplugin;

import com.retroeditor.plugin.Plugin;
import com.retroeditor.plugin.PluginContext;
import javafx.application.Platform;
import javafx.scene.control.Menu;
import javafx.scene.control.MenuItem;

public class MiPlugin implements Plugin {

    private PluginContext context;
    private Menu          miMenu;

    /** Se llama al cargar el JAR. Guarda el contexto para usarlo despues. */
    @Override
    public void onLoad(PluginContext context) {
        this.context = context;
        context.log("MiPlugin cargado correctamente.");
    }

    /** Se llama cuando la UI esta lista. Aqui agregamos nuestros elementos. */
    @Override
    public void onEnable() {
        // Toda modificacion de la UI debe hacerse en el hilo de JavaFX
        Platform.runLater(() -> {
            miMenu = new Menu("Mi Plugin");

            MenuItem saludar = new MenuItem("Saludar");
            saludar.setOnAction(e -> context.log("Hola desde MiPlugin!"));

            miMenu.getItems().add(saludar);
```

```java

            // Registrar el menu: SamaruC lo eliminara automaticamente
            // al desactivar el plugin
            context.registerMenu(miMenu);
        });
    }

    /** Se llama al deshabilitar o cerrar la app. Limpia tus recursos. */
    @Override
    public void onDisable() {
        // Los menus registrados con registerMenu() se limpian automaticamente,
        // pero aqui puedes liberar cualquier otro recurso propio.
        context.log("MiPlugin desactivado.");
    }

    /** Nombre visible en la pantalla de configuracion de SamaruC. */
    @Override
    public String getName() {
        return "Mi Plugin de Ejemplo";
    }
}
```

Listing 1.2: Implementación básica de un plugin

1.6. Registrar el plugin con ServiceLoader

Java ServiceLoader requiere un fichero descriptor en `META-INF/services/`.

Crea el archivo: `src/main/resources/META-INF/services/com.retroeditor.plugin.Plugin`

Con el contenido (nombre completo de tu clase implementadora):

```
com.miplugin.MiPlugin
```

Si tu plugin tiene **varias implementaciones**, ponlas una por línea:

```
com.miplugin.MiPlugin
com.miplugin.OtroPlugin
```

1.7. Usar el `PluginContext`

La interfaz `PluginContext` proporciona los siguientes métodos:

Método	Descripción
`getPrimaryStage()`	Devuelve el `Stage` principal de la aplicación. Útil para crear diálogos modales.
`getScene()`	Devuelve la `Scene` principal. Permite acceder al `root` para añadir nodos.
`getMenuBar()`	Devuelve la `MenuBar` de la app. Puedes añadir menús directamente o usar `registerMenu()`.
`log(String)`	Escribe un mensaje en la pestaña **Monitor** de SamaruC.
`registerMenu(Menu)`	Añade un `Menu` a la barra de menú y registra la referencia para limpieza automática.
`unregisterMenu(Menu)`	Elimina manualmente un menú registrado.
`unregisterAll()`	Elimina todos los elementos registrados por este plugin. Lo llama SamaruC automáticamente.

Tabla 1.2: Métodos de PluginContext

1.7.1. Ejemplo: abrir un diálogo con el Stage del host

```java
@Override
public void onEnable() {
    Platform.runLater(() -> {
        Menu menu = new Menu("Herramientas");
        MenuItem abrirDialogo = new MenuItem("Abrir mi dialogo");
        abrirDialogo.setOnAction(e -> {
            Stage dialogo = new Stage();
            dialogo.initOwner(context.getPrimaryStage());   // modal respecto al host
            dialogo.setTitle("Mi herramienta");
            // ... configurar la escena del dialogo ...
            dialogo.show();
        });
        menu.getItems().add(abrirDialogo);
        context.registerMenu(menu);
    });
}
```

1.7.2. Ejemplo: escribir en el Monitor

```
context.log("Procesando fichero: " + archivo.getName());
```

1.8. Empaquetar el plugin como JAR

Desde la raíz de tu proyecto de plugin, ejecuta:

```
mvn package
```

Esto generará `target/mi-plugin-1.0.0.jar`. Ese es el fichero que debes instalar en SamaruC.

Nota: No uses el plugin `maven-shade` en el JAR del plugin para no duplicar clases del host (JavaFX, etc.). El JAR debe contener **solo** las clases propias del plugin.

1.9. Instalar el plugin en SamaruC

Tienes dos formas de instalar un plugin:

1.9.1. Opción A — Desde la interfaz de SamaruC (recomendado)

1. Abre SamaruC Editor.
2. Ve a **Archivo / Configuración** (o el atajo de teclado correspondiente).
3. Selecciona la pestaña **Plugins**.
4. Haz clic en **Instalar plugin** y selecciona tu fichero `.jar`.
5. SamaruC copiara el JAR a la carpeta `plugins/` junto al ejecutable.
6. Pulsa **Recargar plugins** para activarlo sin reiniciar.

1.9.2. Opción B — Copiado manual

Copia el fichero `.jar` directamente en la carpeta `plugins/` que se encuentra en el directorio de trabajo de SamaruC (al lado del ejecutable `RetroEditor.exe` o del JAR principal):

```
RetroEditor/
+-- RetroEditor.exe
+-- app/
|   +-- retro-editor-1.0-SNAPSHOT.jar
+-- plugins/              <- aqui va tu JAR
    +-- mi-plugin-1.0.0.jar
```

Reinicia la aplicación y el plugin se cargará automáticamente.

1.10. Habilitar y deshabilitar plugins

En la pestaña **Plugins** de la pantalla de configuración puedes:

- Ver la lista de todos los JARs descubiertos en `plugins/`.
- Ver su estado (habilitado / deshabilitado).
- Pulsar **Habilitar** o **Deshabilitar** para cambiar el estado.
- Pulsar **Recargar plugins** para aplicar cambios sin reiniciar.

El estado de cada plugin se guarda en el fichero:

```
~/.retroeditor.plugins.properties
```

(donde ~ es tu carpeta de usuario del sistema operativo)

1.11. Ejemplo completo

A continuación se muestra un plugin mínimo funcional que añade un menú **“Utilidades”** con una opción que muestra la ruta del archivo actualmente abierto.

1.11.1. Estructura del proyecto

```
plugin-utilidades/
+-- pom.xml
+-- src/main/
    +-- java/com/ejemplo/UtilsPlugin.java
    +-- resources/META-INF/services/com.retroeditor.plugin.Plugin
```

1.11.2. UtilsPlugin.java

```java
package com.ejemplo;

import com.retroeditor.plugin.Plugin;
import com.retroeditor.plugin.PluginContext;
import javafx.application.Platform;
import javafx.scene.control.*;

public class UtilsPlugin implements Plugin {

    private PluginContext ctx;

    @Override
    public void onLoad(PluginContext context) {
        this.ctx = context;
    }

    @Override
    public void onEnable() {
        Platform.runLater(() -> {
            Menu menuUtils = new Menu("Utilidades");

            MenuItem infoItem = new MenuItem("Informacion del entorno");
            infoItem.setOnAction(e -> {
                String info = "Java: "    + System.getProperty("java.version")
                            + "\nOS: "   + System.getProperty("os.name")
                            + "\nDir: "  + System.getProperty("user.dir");
                ctx.log(info);

                Alert alert = new Alert(Alert.AlertType.INFORMATION, info, ButtonType.OK);
```

```
                alert.setTitle("Entorno");
                alert.initOwner(ctx.getPrimaryStage());
                alert.showAndWait();
            });

            menuUtils.getItems().add(infoItem);
            ctx.registerMenu(menuUtils);
        });
    }

    @Override
    public void onDisable() {
        ctx.log("UtilsPlugin desactivado.");
    }

    @Override
    public String getName() {
        return "Plugin de Utilidades";
    }
}
```

1.11.3. META-INF/services/com.retroeditor.plugin.Plugin

```
com.ejemplo.UtilsPlugin
```

1.11.4. Compilar e instalar

```
cd plugin-utilidades
mvn package
copy target\plugin-utilidades-1.0.0.jar ..\RetroEditor\plugins\
```

1.12. Buenas prácticas y limitaciones

1.12.1. Buenas prácticas

- **Siempre usa** `Platform.runLater()` para cualquier modificación de la UI de JavaFX desde métodos del ciclo de vida del plugin.
- **Limpia tus recursos en** `onDisable()`: cierra hilos, streams, conexiones de red, etc.
- **Usa** `context.registerMenu()` en lugar de acceder directamente a `getMenuBar()` `.getMenus().add()` para que SamaruC pueda limpiar automáticamente al desactivar el plugin.
- **Declara** `provided` todas las dependencias que ya provee el host (JavaFX, SamaruC, etc.).
- **Versiona tu plugin** y sigue convenciones de nombre de paquete para evitar conflictos de clases.

1.12.2. Limitaciones actuales

- Los plugins **no pueden reemplazar** el editor de código ni el árbol de proyecto (no existen puntos de extensión para esos componentes todavía).
- No hay sistema de dependencias entre plugins: si tu plugin depende de otro, ambos JARs deben estar presentes y el orden de carga no está garantizado.
- El classloader usa estrategia **child-first**, por lo que puedes incluir versiones propias de librerías en tu JAR; sin embargo, las clases de `java.*`, `javax.*`, `javafx.*` y `org.kordamp.*` siempre se cargan desde el host.
- Los plugins se cargan **desde el directorio de trabajo** (`plugins/` junto al ejecutable), no desde rutas arbitrarias del sistema.
- Actualmente no existe una API para acceder al contenido del editor de texto abierto. Esto se planea para futuras versiones.